

Architectural Blueprint and Prompt Engineering Framework for Industrial-Grade Agentic AI Platforms

The convergence of the Industrial Internet of Things (IIoT), legacy Manufacturing Execution Systems (MES), and generative artificial intelligence represents a foundational paradigm shift in operational technology. The deployment of autonomous, agentic artificial intelligence within industrial environments necessitates an architectural foundation that balances rapid innovation with stringent security, scalability, and cost-efficiency. Manufacturers frequently operate heterogeneous digital environments comprising dozens of disparate MES, Supervisory Control and Data Acquisition (SCADA), and Enterprise Resource Planning (ERP) versions, each harboring unique database schemas and proprietary APIs¹. Constructing a monolithic AI integration that hardcodes SQL queries or relies on inflexible REST integrations inevitably fails upon the first system upgrade or cross-site deployment.

To circumvent this fragility, the deployment of a universal, version-agnostic agentic AI platform requires a highly modular, decoupled architecture. This analysis deconstructs the requirements for building such an industrial-grade AI platform, structurally segmented into a React-based client interface (`mai_frontend`) and a FastAPI-driven server-side orchestration layer (`mai_backend`). The intelligence of the system is governed by advanced AI frameworks, including LangChain, LangGraph, and LiteLLM, while the perimeter is secured by multi-tiered cybersecurity guardrails explicitly designed to neutralize the Open Worldwide Application Security Project (OWASP) Top 10 vulnerabilities for Large Language Model (LLM) applications². The synthesis of these architectural, operational, and security imperatives culminates in a highly structured, expert-level prompt designed to instruct an autonomous AI coding agent to construct the entire project from the ground up.

Frontend UI and UX Architecture: The Command Center Interface

The user interface for an industrial intelligence platform must distill vast streams of operational telemetry, predictive AI forecasts, and critical safety alerts into an accessible, actionable, and visually coherent format. The foundational hyper-text markup language (HTML) simulation establishes a clear visual and navigational hierarchy that must be systematically refactored into a scalable, component-driven React architecture within the `mai_frontend` directory⁴.

The primary structural paradigm comprises a persistent sidebar for application-wide navigation, a top context bar for persona toggling and global state visibility, and a dynamic main viewport that conditionally renders distinct functional modules based on user interaction⁴. In a React environment, this necessitates a root layout component wrapping a robust routing system. The sidebar navigation dictates the primary route structure, encompassing the "Command Centre Overview," "Use-Case Library," "AI Agents," "Admin Console," "Analytics &

ROI," and "Platform Settings"⁴. State management represents a critical architectural consideration. The application must maintain global states for the currently selected persona, such as a "Manufacturing Head" focused on production balancing versus a "Digital & IT Head" focused on governance and connector health⁴. Toggling these personas alters the visibility, opacity, and emphasis of specific dashboard elements, requiring a centralized state management solution like Zustand or React Context to propagate these changes without excessive prop-drilling⁴.

Real-Time Telemetry and Operational Dashboards

The command center overview is highly modular, suggesting a grid-based layout utilizing modern CSS Grid or Tailwind CSS utility classes within the React components. Key Performance Indicator (KPI) tiles display real-time metrics, including Production Output (e.g., throughput versus plan), Plant Overall Equipment Effectiveness (OEE), Zero Harm Index for safety performance, and Revenue Year-to-Date⁴. These tiles require asynchronous data fetching mechanisms, leveraging libraries like React Query to manage loading states, caching, and background polling.

A central operational feature is the "Live Agent Activity" feed, simulating a continuous stream of AI-generated events that vary in severity from routine system updates to critical equipment failure predictions⁴. Implementing this in React requires establishing WebSocket connections or Server-Sent Events (SSE) to receive real-time updates from the FastAPI backend, seamlessly updating a local state array that dictates the rendered list with animated entry transitions. The adjacent "AI Executive Summary" block further synthesizes operational data into categorized, actionable insights: "Requires Action," "Decisions this week," and "Positive Signals"⁴. This necessitates structured JSON responses from the backend, which the frontend iteratively maps to distinct visual alert components.

Modular Capabilities: The Use-Case Library and Agent Ecosystem

The platform's functional breadth is encapsulated within a Use-Case Library, featuring 23 distinct deployable modules categorized into five strategic pillars and filterable by technological tags. The React implementation must manage complex, multi-variable filtering states efficiently, dynamically recalculating the displayed grid of use-case cards.

Strategic Pillar	Focus Area	Representative AI Use Cases
Operational Excellence	Daily reporting, SOP compliance, insights.	Agentic Daily Operations Reporting, Chat with Data, Intelligent Document Extraction ⁴ .
Sustainable Operations	Safety, environmental	Computer Vision Safety & Site Intelligence, AI

	monitoring, asset health.	Spill/Leak Detection, IoT Asset Tracking ⁴ .
Smart Growth & Expansion	Planning, scheduling, digital twins.	Agentic Production Orchestration, Digital Twin What-If Analytics ⁴ .
Financial Resilience	Threat intelligence, anomaly detection, forecasting.	OT/ICS Cyber-Anomaly Detection, Commercial Forecast Intelligence, Supply Chain Risk ⁴ .
Horizontal Enterprise	HR, procurement, technical support, finance.	AI Technical Support Desk, Finance Invoice & Budget Control, HR Candidate Assessment ⁴ .

Detailed modal overlays triggered by interacting with a use-case card demand a centralized modal context system (e.g., React Portals) to ensure proper z-indexing, background blurring, and accessibility compliance. These modals display the business impact, rollout status, and granular administrative controls for each module, reflecting the platform's capability-driven design¹. Furthermore, the platform explicitly delineates 12 specialized, always-on AI agents—ranging from an Operations Agent monitoring throughput to a Predictive Safety Intelligence Agent forecasting heat stress and equipment failure—each requiring individual enablement toggles and configuration interfaces within the "AI Agents" gallery⁴.

Governance and Administrative Controls

The Admin Console introduces complex data tables displaying user roles, site connectivity statuses, integration health, and alert escalation rules⁴. To maintain an industrial-grade user experience, these tables must support advanced sorting, pagination, and inline status badges. The platform's governance model is highly granular, supporting Role-Based Access Control (RBAC) with personas such as Super Admin, Plant Digital Head, Operations Head, and HSE Officer⁴. The React frontend must securely consume RBAC profiles from the backend, conditionally rendering administrative panes—such as "Users & Roles," "Sites & Plants," "Governance & Guardrails," and "Notifications"—based on the authenticated user's authorization level¹.

Backend Architecture: FastAPI and Universal Integration Strategy

The server-side infrastructure, designated for the `mai_backend` directory, must be engineered using FastAPI to leverage its high-performance asynchronous event loop, native type hinting, and automatic OpenAPI documentation generation⁵. An industrial AI platform transcends

simple database record manipulation; it orchestrates long-running, stateful agentic workflows, interfaces with fragile legacy enterprise systems, and manages continuous IoT telemetry ingestion¹.

FastAPI's architecture should be meticulously compartmentalized using APIRouter instances: a core router for handling standard CRUD operations regarding users, RBAC, and configuration; a telemetry router managing WebSocket connections for live dashboard updates; and a dedicated AI orchestration router triggering complex LangGraph pipelines. Dependency injection, a foundational feature of FastAPI, must be utilized to manage database sessions, authentication contexts, and connection pools to external systems, ensuring clean separation of concerns and testability⁵.

The Canonical Data Model and Version-Agnostic Connectivity

The central engineering challenge in industrial AI deployments is the extreme variability of customer data schemas. The backend database schema must adhere to a schema-first, version-agnostic philosophy. A conventional integration approach—hard-coding SQL queries directly against a specific instance of SAP ERP or a localized SCADA system—is inherently brittle¹. Instead, the backend must implement a Canonical Data Model, an internal, vendor-neutral representation of manufacturing concepts such as `work_order`, `machine_state`, `inventory_lot`, and `safety_incident`¹.

The platform achieves universal integration through three distinct approaches, dynamically selected per module and per customer environment:

Integration Approach	Mechanism	Optimal Use Case
Approach A: Schema Registry	Lightweight, read-only extraction of metadata (table names, columns, data types) without touching row data.	Customers unwilling to expose backend APIs; enables rapid auto-population of the platform's Capability Matrix ¹ .
Approach B: Direct Backend API	Registration of customer REST/SOAP/OData endpoints or message buses, facilitated by Protocol Adapters.	Preferred path; respects the customer's native business logic and validation layers, particularly for write-back actions ¹ .
Approach C: Scoped Database Access	Read-only (and narrowly scoped write) database credentials limited to an explicitly agreed upon table	Environments lacking modern APIs but permitting direct data warehousing

	allow-list.	access ¹ .
--	-------------	-----------------------

This architecture powers the "Capability Resolver." When the platform ingests a new customer schema, an LLM-assisted "Schema Interpreter" proposes a mapping to the Canonical Data Model¹. Once a human administrator confirms this mapping, the Capability Resolver evaluates every agent's manifest. If an agent requires canonical fields that are successfully mapped, it appears as "Available" in the Use-Case Library; if required fields are missing, the use case is automatically hidden rather than generating runtime errors¹. This ensures the frontend UI strictly reflects the actual capabilities of the underlying legacy systems.

Persistent State and Task Orchestration

For persistent state management, PostgreSQL serves as the primary system of record, storing the Schema Registry, RBAC policies, encrypted credentials, and the extensive audit logs required for industrial compliance¹. This relational foundation is augmented by vector extensions, such as pgvector, enabling high-performance semantic search and Retrieval-Augmented Generation (RAG) over unstructured text, including safety manuals and Standard Operating Procedures (SOPs)¹. High-frequency telemetry from IoT sensors—monitoring asset vibration, temperature, and energy draw—necessitates a time-series approach, optimizing storage and query performance for temporal data¹. To prevent the asynchronous FastAPI application from blocking during computationally intensive LLM processing or complex data aggregation tasks, the architecture must implement a distributed task queue¹. Leveraging Celery backed by Redis allows the backend to acknowledge a user request immediately, offload the multi-step agentic reasoning to a background worker pool, and asynchronously push the final result to the React frontend via WebSocket upon completion, ensuring a highly responsive user experience even under heavy load.

Agentic AI Engine: LangGraph and LiteLLM Orchestration

The intelligence of the platform is driven by a sophisticated orchestration layer that coordinates multiple language models, deterministic tools, and enterprise data sources. While basic chatbot implementations rely on linear, conversational chains, industrial automation requires the robustness of stateful, multi-agent workflows. LangGraph emerges as the preeminent framework for this architectural requirement, providing the necessary infrastructure to define inspectable, debuggable, and recoverable agent execution paths⁸.

Stateful Graph Execution via LangGraph

LangGraph models AI execution as a cyclical state machine, where discrete operational logic is encapsulated within nodes, and the transition between these operations is governed by conditional edges⁸. This paradigm is vastly superior to black-box LLM chains because it allows developers to define explicit failure recovery mechanisms and human-in-the-loop interruption

points⁸.

Consider the implementation of an advanced "Data Extraction and Reporting Agent." The workflow initiates with a strongly typed AgentState dictionary, which travels through the graph, accumulating user intent, database context, generated queries, execution results, and error flags¹⁰. The execution flow proceeds through specific nodes:

1. **Intent Understanding Node:** Utilizes a high-speed LLM to parse the natural language query, extracting the requested report type, target metrics, and temporal filters into a structured JSON object¹⁰.
2. **Schema Context Retrieval Node:** Queries the PostgreSQL Schema Registry to fetch the allowed canonical database tables and column metadata relevant to the user's intent¹⁰.
3. **SQL Generation Node:** An LLM processes the user intent and the permitted schema to generate read-only T-SQL SELECT queries, strictly avoiding destructive operations¹⁰.
4. **SQL Execution Node:** The backend attempts to execute the generated query against the mapped legacy database.
5. **Conditional Routing & Self-Correction:** This is where LangGraph excels. If the SQL Execution Node encounters a syntax error or a missing column exception, a conditional edge evaluates the `retry_count`. If under the threshold, the graph routes the state back to the SQL Generation Node, injecting the specific database error message as context for a self-correction attempt⁹.
6. **Data Profiling and Insights Nodes:** Successful data extraction routes to profiling nodes that clean the payload, followed by an analysis node that relies on a frontier reasoning model to generate an executive summary and corrective action recommendations based strictly on the retrieved data¹⁰.

This explicit state management ensures that failures are legible. When a tool fails, the system does not silently crash or hallucinate a response; it systematically attempts recovery or routes to a defined fallback node, documenting the exact point of failure in the audit log⁸.

Model Routing and Cost Optimization via LiteLLM

Industrial enterprises cannot risk vendor lock-in with a single LLM provider. The platform implements LiteLLM to serve as a unified, provider-agnostic gateway⁵. LiteLLM standardizes API calls across diverse models—including OpenAI's GPT-4, Anthropic's Claude, Google's Gemini, and self-hosted open-weight models like Llama 3 or Mistral—allowing the backend code to remain completely agnostic to the underlying provider⁵.

This gateway architecture is critical for implementing intelligent fallback routing and cost optimization. If a primary model experiences a provider-side outage or triggers a rate limit, LiteLLM automatically intercepts the error and routes the request to the next configured model in the fallback chain without disrupting the LangGraph execution¹. Furthermore, the platform utilizes cost-tier selection based on task complexity. Simple intent classification or data extraction tasks are routed to fast, inexpensive models (e.g., Llama 3 8B or Gemini Flash), while complex, unstructured reasoning tasks are reserved for premium frontier models, drastically reducing the total token expenditure⁵. LiteLLM tracks the token usage and cost per request, associating it with the specific agent and customer tenant, feeding this data directly into the

React frontend's Analytics & ROI dashboard¹.

Human-in-the-Loop (HITL) and Autonomous Execution

The deployment of AI in manufacturing requires a Human-in-the-Loop paradigm by design¹. Autonomous execution of actions carrying operational, safety, or financial risk—such as altering a production schedule, authorizing a purchase order, or modifying a control system parameter—is fundamentally unacceptable. LangGraph natively supports HITL by utilizing durable checkpointing⁸.

When an agent graph reaches a node designated as a high-risk action, it persists its current state to the database and halts execution. The FastAPI backend emits a notification to the React frontend or via MS Teams/SMS. A human administrator (e.g., a Shift Supervisor or Plant Head) reviews the agent's proposed action, the reasoning trace, and the projected impact. Only upon explicit human approval does the backend send a resumption signal to LangGraph, allowing the workflow to proceed from the exact checkpoint and execute the tool¹. This capability transforms the AI from an uncontrollable black box into an auditable, advisory co-pilot.

Cybersecurity Architecture: Defending the AI Perimeter

Integrating non-deterministic language models into critical Operational Technology (OT) and IT networks introduces a novel and highly complex attack surface. Conventional web application firewalls and perimeter defenses are insufficient to protect against the vulnerabilities inherent in prompt-based interactions. The architecture must systematically address the OWASP Top 10 for LLM Applications—specifically Prompt Injection, Insecure Output Handling, Excessive Agency, and Sensitive Information Disclosure—embedding defensive mechanisms deeply into the request lifecycle².

Mitigating Prompt Injection and Jailbreaks

Prompt Injection (OWASP LLM01) constitutes the most severe threat vector. It occurs when malicious input circumvents the system's foundational instructions, coercing the model into generating harmful content, executing unauthorized actions, or exfiltrating data³. This vulnerability manifests in two primary forms: Direct Prompt Injection (jailbreaking), where an attacker explicitly instructs the model to ignore prior commands, and Indirect Prompt Injection, where the model ingests poisoned data from an external source—such as a manipulated PDF document or an altered supplier website—effectively turning the LLM into a "confused deputy" acting on behalf of the attacker¹².

Relying solely on defensive instructions within the system prompt is demonstrably ineffective, as sophisticated adversarial inputs routinely bypass these semantic guardrails. To robustly mitigate this risk, the architecture employs an out-of-process AI firewall, leveraging frameworks like NeMo Guardrails or Guardrails AI¹³. This firewall operates as an independent microservice or is integrated as a global FastAPI middleware, intercepting every user request before it reaches the LangGraph orchestration layer, and analyzing every response before it is

transmitted to the user¹⁴.

The firewall implements a rigorous, multi-tiered defensive strategy:

Defense Tier	Technological Mechanism	Target Threat Profile	Operational Characteristics
Tier 0: Deterministic Blocking	Regular expressions (Regex), exact string matching, abstract syntax trees (AST), checksum validation.	Explicit Personally Identifiable Information (PII), known jailbreak signatures, OS command injection strings (e.g., <code>rm -rf</code> , <code>DROP TABLE</code>).	Near-zero latency; blocks clear, unambiguous violations instantly without incurring LLM inference costs. Highly auditable ¹⁴ .
Tier 1: Semantic Context Classifier	Fine-tuned Small Language Models (SLMs) or constrained LLM-as-a-judge forcing discrete labels (e.g., allow/block/escalate).	Contextual prompt injection, subtle adversarial suffixes, topical deviation, roleplay manipulation.	Minor latency penalty; evaluates the semantic intent of the payload rather than just the syntax, catching sophisticated attacks ¹⁴ .
Tier 2: Policy & Persona Engine	Role-Based Access Control (RBAC) thresholds cross-referenced with semantic classification.	Unauthorized data access, internal privilege escalation.	Adjusts the false-positive/false-negative ratio dynamically. Enforces that a shop-floor operator is blocked from querying executive financial forecasts ¹⁴ .
Tier 3: Human Escalation	Checkpointing and routing to high-friction human review workflows.	Borderline inputs, uncertain semantic classifications, high-risk operational	Intentionally sacrifices UX speed for safety; ensures critical ambiguity is resolved by

		commands.	authorized personnel ¹⁴ .
--	--	-----------	--------------------------------------

Insecure Output Handling and Data Poisoning

Insecure Output Handling (OWASP LLM02) arises when downstream backend components blindly accept and execute LLM-generated output without proper sanitization or validation³. If an agent is tasked with generating a SQL query based on a natural language request, executing that unstructured string directly against the database exposes the entire infrastructure to catastrophic SQL injection, Remote Code Execution (RCE), or Server-Side Request Forgery (SSRF)³.

The platform neutralizes this risk by strictly enforcing output schemas utilizing Pydantic v2 and Guardrails AI¹¹. Before any LLM response is passed to a LangGraph execution node, it must conform to a predefined, statically typed data structure. If the model attempts to output arbitrary code, unexpected data types, or attempts to hallucinate a response outside the permitted schema, the validation layer intercepts the anomaly, raises an exception, and safely aborts the operation or triggers a self-correction loop¹¹.

Training Data Poisoning (OWASP LLM03) represents a supply-chain vulnerability where malicious actors introduce manipulated data during fine-tuning or embedding generation³. In an industrial context, if a threat actor alters an equipment maintenance manual stored in the vector database, the RAG system may instruct technicians to perform dangerous, non-compliant procedures. Mitigation requires tracking data provenance, rigorously validating document ingestion pipelines, and ensuring the vector database (pgvector) is subjected to the same strict access controls as the primary relational database¹⁸.

Excessive Agency and OT Cyber-Anomaly Detection

Excessive Agency (OWASP LLM06) occurs when an LLM possesses permissions, functionality, or autonomy exceeding what is strictly necessary for its intended purpose². The platform mitigates this by applying the principle of least privilege to all tools integrated via LangChain. A data retrieval agent is equipped solely with localized SELECT capabilities on specific canonical views, entirely lacking the functionality to modify state, delete records, or access the external internet¹⁰. The mandatory Human-in-the-Loop checkpointing acts as the ultimate circuit breaker for tools that require write access, ensuring that no state-altering action occurs without cryptographic, auditable human authorization¹.

Beyond protecting the LLM itself, the platform actively utilizes AI for enterprise defense through the OT/ICS Cyber-Anomaly Detection agent¹. This module continuously monitors the Programmable Logic Controller (PLC) network and SCADA traffic. By utilizing unsupervised machine learning to baseline normal operational behavior, the agent can instantly detect and flag anomalous industrial device commands or unusual telemetry patterns, providing early warning of cyber interference before it endangers physical production or safety¹.

Infrastructure, Scalability, and Deployment Topologies

To achieve the reliability required for continuous manufacturing operations, the deployment architecture must support an edge-hybrid topology¹. Relying entirely on cloud-based infrastructure introduces network latency and data sovereignty challenges that are unacceptable for critical Operational Technology. By containerizing the FastAPI backend, React frontend, and Redis message broker using Docker, and orchestrating them via Kubernetes (e.g., GKE or lightweight K3s for edge environments), the platform achieves deployment portability¹. In the edge-hybrid model, latency-sensitive agents—such as the Computer Vision Safety Agent monitoring local CCTV feeds for PPE violations—are deployed directly onto edge appliances within the manufacturing facility¹. These edge nodes execute inference using highly optimized, quantized open-weight models (e.g., deployed via Ollama or vLLM), eliminating cloud round-trip times and preventing the transmission of continuous, high-bandwidth video streams over external networks²⁰. Conversely, computationally intensive, non-real-time agents—such as the Commercial Forecast Intelligence module—operate in the centralized cloud, leveraging frontier models via the LiteLLM gateway to aggregate data across multiple global plants¹. API keys for external LLM providers are never stored in plaintext. The backend utilizes envelope encryption, securing the API keys via a Data Encryption Key (DEK) backed by a HashiCorp Vault or a local encrypted keystore¹. This ensures that even in the event of a database breach, the highly privileged credentials granting access to external AI services remain protected.

The Comprehensive Prompt Engineering Framework

The culmination of this architectural analysis is the formulation of a highly precise, multi-dimensional prompt. The objective is to instruct an autonomous AI coding agent to initialize, structure, and populate the dual-repository project (mai_frontend and mai_backend), integrating all specified technologies while strictly adhering to the architectural, operational, and security paradigms delineated above.

The prompt utilizes explicit structural markers, constraint directives, and cybersecurity mandates to restrict the AI's generation path, forcing the application of industrial best practices rather than relying on insecure, default boilerplate code.

System Prompt for Autonomous AI Agent Initialization

[BEGIN PROMPT]

Role and Execution Directive

You are a Senior Principal AI Systems Architect and Cybersecurity Expert specializing in Industrial IoT (IIoT) platforms. Your explicit objective is to scaffold, design, and implement a complete, industrial-grade, cost-effective full-stack project named "Manufacturing Agentic AI" (MAI). You will act autonomously to generate all necessary folder structures, configuration files, and core application code. You must ensure strict adherence to modern software engineering, asynchronous processing, and defensive cybersecurity practices.

Project Architecture and Technology Stack

The project must be strictly divided into two root directories: mai_frontend and mai_backend.

1. mai_frontend (Client-Side Interface):

- **Framework:** React (using Vite for initialization) with TypeScript.
- **Styling:** Tailwind CSS for rapid, scalable styling matching a high-density industrial command center aesthetic (utilizing deep navies, teals, and slate grays based on provided HTML visual cues).
- **State Management:** Zustand for synchronous global state (e.g., persona toggling between 'Manufacturing Head' and 'IT Head', sidebar expansion) and React Query for asynchronous data fetching and caching.
- **UI Requirements:** Based on an industrial command center layout. You must implement:
 - Sidebar.tsx: A persistent navigation menu.
 - Topbar.tsx: A context bar with a Persona toggle switch that updates global state.
 - KPIDashboard.tsx: A grid layout displaying real-time metrics (Production Output, OEE).
 - LiveFeed.tsx: A WebSocket consumer component that renders an animated, auto-updating list of AI-generated alerts and events.
 - UseCaseLibrary.tsx: A complex grid component that renders use-case cards, including interactive filters for Strategic Pillars (e.g., Operational Excellence) and Tech Tags (e.g., GENAI, VISION).

2. mai_backend (Server-Side Logic and AI Orchestration):

- **Framework:** FastAPI (Python 3.11+) utilizing asynchronous routing (async def) and Pydantic v2 for rigorous data validation.
- **Database:** PostgreSQL (augmented with pgvector for embedding storage) managed via SQLAlchemy (using the async engine) and Alembic for schema migrations.
- **AI Frameworks & Tooling:**
 - **LangGraph:** Must be used to define stateful, cyclical multi-agent workflows.
 - **LangChain:** For structured tool creation, document loading, and prompt templating.
 - **LiteLLM:** Must be implemented as the centralized LLM gateway to handle routing, fallback mechanisms, load balancing, and cost tracking across multiple providers (e.g., OpenAI, Anthropic, local Ollama models).
 - **Guardrails AI / NeMo Guardrails:** Implemented to enforce strict output validation and input sanitization.

Core Implementation Mandates & Security Requirements

Mandate A: Agentic Workflow Design (LangGraph)

Within mai_backend, create a dedicated agents/ directory. You must implement a production-ready, stateful LangGraph workflow representing a "Data Extraction and Reporting Agent."

- Define a robust TypedDict for the graph state that tracks intent, SQL queries, row counts, and error messages.
- Implement distinct functional nodes: understand_intent, fetch_schema_context, generate_sql_query, execute_query_safe, and compile_insights.
- **Crucial:** Implement conditional routing logic (edges). If generate_sql_query creates invalid SQL, the graph must route back to the generation node, passing the database error as context, restricted by a MAX_RETRIES counter to prevent infinite loops.

- Implement a Human-in-the-Loop (HITL) checkpoint. If an action requires writing to the database, the graph must pause execution, requiring a specific API call to resume.

Mandate B: Defensive Cybersecurity Architecture (OWASP LLM Mitigation)

You must embed rigorous security controls addressing the OWASP Top 10 for LLM Applications within the FastAPI backend:

1. **Prompt Injection Defense (LLM01):** Implement a global FastAPI middleware interceptor or an out-of-process firewall utilizing Guardrails AI. This layer must scan incoming requests for malicious intents, jailbreak signatures, or OS command injection patterns before they are processed by LangGraph.
2. **Insecure Output Handling (LLM02):** Enforce that all LLM outputs utilized for backend execution are strictly validated through Pydantic v2 models. Unstructured text must *never* be passed directly to a database execution function or system shell.
3. **Excessive Agency (LLM06):** Ensure that all tools provided to the LangChain agents operate on the principle of least privilege. Database tools must be strictly scoped to SELECT operations on specific canonical views.
4. **API Security:** Implement JWT-based authentication for all FastAPI routes and secure CORS configurations restricting access exclusively to the mai_frontend origin.

Mandate C: Cost Effectiveness, Reliability, and Fallbacks

Configure the LiteLLM gateway within the backend to demonstrate industrial reliability. Define a fallback array: if a primary frontier model (e.g., gpt-4o) times out or encounters an API rate limit, the LiteLLM gateway must automatically and silently degrade to a smaller, faster model (e.g., gemini-1.5-flash or a local llama3) without failing the overarching FastAPI request.

Execution Steps for the AI Generation:

1. **Project Initialization:** Output the exact terminal commands required to set up the mai_frontend (Vite) and mai_backend (Python virtual environment) folders.
2. **Backend Core:** Generate the code for main.py (FastAPI initialization, CORS, Middleware), llm_gateway.py (LiteLLM configuration and routing), guardrails_firewall.py (Input/Output sanitization logic), and agent_workflow.py (The complete LangGraph state machine implementation).
3. **Frontend Core:** Generate the code for the main layout component (DashboardLayout.tsx), the global state store (store.ts using Zustand), and the WebSocket consumer component for the Live Feed (LiveFeed.tsx).
4. **Documentation:** Provide a structured README.md explaining the architecture and providing instructions on how to run both the frontend and backend services concurrently using Docker Compose.

Generate the codebase adhering strictly to these architectural boundaries, focusing on depth, security, and industrial viability. Do not output trivial "hello world" examples; output production-ready structural code.

[END PROMPT]

Conclusion

The transition from monolithic industrial software to an agentic AI ecosystem requires a profound architectural reimagining. By enforcing strict separation of concerns—coupling a

reactive, high-density React frontend with an asynchronous, securely orchestrated FastAPI backend—organizations can achieve the agility of modern web applications without compromising the safety and reliability demanded by Operational Technology¹. The integration of LangGraph provides the necessary stateful control to ensure AI agents act predictably, recovering gracefully from errors and deferring to human oversight for critical actions⁸. Simultaneously, the LiteLLM gateway abstracting the model layer guarantees cost-efficiency and prevents vendor lock-in⁵. However, the viability of this platform ultimately hinges on its cybersecurity posture. By proactively anticipating and mitigating the OWASP Top 10 vulnerabilities for LLMs through a multi-tiered firewall and rigorous output validation, the platform transforms a potentially hazardous technology into a secure, value-generating industrial asset³. The provided prompt operationalizes these principles, providing a deterministic blueprint for the automated construction of this advanced intelligence architecture.

Works cited

1. blueprint.docx
2. What are the OWASP Top 10 risks for LLMs? | Trend Micro (US), https://www.trendmicro.com/en_us/what-is/ai/owasp-top-10.html
3. OWASP Top 10 LLM Vulnerabilities & Checklist (2026) - Lasso Security, <https://www.lasso.security/blog/owasp-top-10-llm-vulnerabilities-security-checklist>
4. IIoT_Manufacturing_Agentic_AI_Platform_Simulation.html
5. Loblaws: Building Alfred: Production-Ready Agentic Orchestration Layer for E-commerce - ZenML LLMops Database, <https://www.zenml.io/llmops-database/building-alfred-production-ready-agentic-orchestration-layer-for-e-commerce>
6. pr1m8/pjkm: Python project scaffolder for humans and agents — 105 groups, 22 recipes, MCP server, group-aware templates - GitHub, <https://github.com/pr1m8/pjkm>
7. Generative AI & Conversational AI - RAG, chatbots, document intelligence - AiSPRY, <https://www.aispry.ai/solutions/generative-ai>
8. Comparing Open-Source AI Agent Frameworks in 2026 - Future AGI, <https://futureagi.com/blog/oss-agent-frameworks-2026/>
9. LangGraph - Reddit, <https://www.reddit.com/r/LangGraph/rising/>
10. readonly_agent_report.ipynb
11. Full Stack GenAI Roadmap 2026: Beginner to Interview Ready Developer, <https://genai-roadmap.purnendudas.in/>
12. OWASP Top 10 for LLM Applications, https://owasp.org/www-project-top-10-for-large-language-model-applications/assets/PDF/OWASP-Top-10-for-LLMs-2023-v1_0_1.pdf
13. Programmable Safety Rails: NVIDIA NeMo Guardrails and TrueFoundry AI Gateway, <https://www.truefoundry.com/blog/nvidia-nemo-guardrails-truefoundry-ai-gatew>

- [ay](#)
14. llm_firewall_1.md · John6666/forum3 at main - Hugging Face,
https://huggingface.co/datasets/John6666/forum3/blob/main/llm_firewall_1.md
 15. Multi-Agent Firewall Architecture for Privacy Protection of Sensitive Data in Interactions with Language Models - arXiv, <https://arxiv.org/html/2607.08282v1>
 16. OWASP Top 10 for LLM Applications v1.0.1,
<https://owasp.org/www-chapter-cincinnati/assets/presentations/OWASP-Top-10-for-LLM-Applications-Deck-v101-3.pdf>
 17. AI Guardrails — Production LLM Safety Guide (2026) - MyEngineeringPath,
<https://myengineeringpath.dev/genai-engineer/ai-guardrails/>
 18. OWASP Top 10 for LLM Applications 2025 - WorldTech IT,
<https://wtit.com/blog/2025/04/17/owasp-top-10-for-llm-applications-2025/>
 19. LLMops in Production: Another 419 Case Studies of What Actually Works - ZenML Blog,
<https://www.zenml.io/blog/llmops-in-production-another-419-case-studies-of-what-actually-works>
 20. An awesome & curated list of best LLMops tools for developers - GitHub,
<https://github.com/tensorchord/Awesome-LLMOps>